



TEXAS A&M UNIVERSITY - SAN ANTONIO

CSCI 2436 Data Structures

Chapter 5 Stacks

Spring 2026

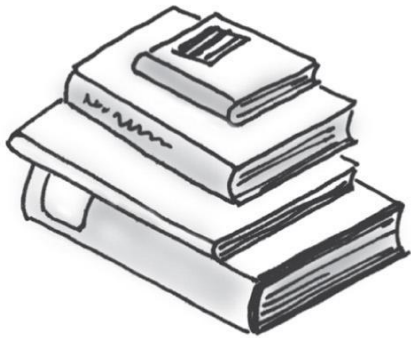


Objectives

- Understand a stack data structure
- Use a stack to convert an infix expression to a postfix expression
- Use a stack to evaluate a postfix expression
- Use a stack to decide whether the delimiters in an algebraic expression are paired correctly
- Use a stack in a program



Stack



© 2019 Pearson Education, Inc.

- Add item on top of stack
- Remove item from top of stack
- Last In First Out (**LIFO**)





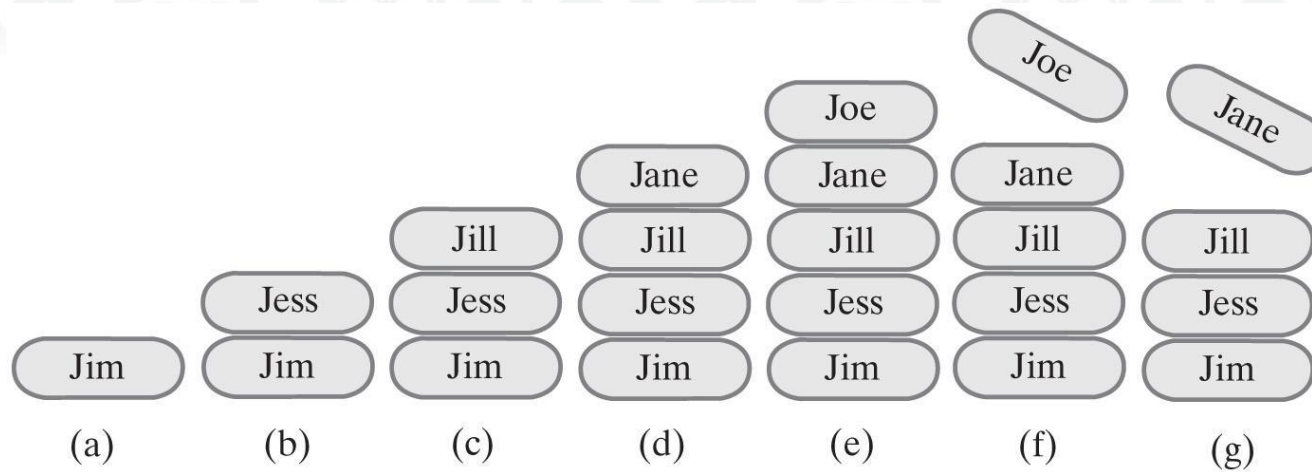
Specifications of the Stack

Data: A collection of objects in **reverse chronological** order and having the **same data type**

Pseudocode	UML	Description
push(newEntry)	+push(newEntry: T): void	Task: Adds a new entry to the top of the stack. Input: newEntry is the new entry. Output: None.
pop()	+pop(): T	Task: Removes and returns the stack's top entry. Input: None. Output: Returns the stack's top entry. Throws an exception if the stack is empty before the operation.
peek()	+peek(): T	Task: Retrieves the stack's top entry without changing the stack in any way. Input: None. Output: Returns the stack's top entry. Throws an exception if the stack is empty.
isEmpty()	+isEmpty(): boolean	Task: Detects whether the stack is empty. Input: None. Output: Returns true if the stack is empty.
clear()	+clear(): void	Task: Removes all entries from the stack. Input: None. Output: None.



Example of a Stack



© 2019 Pearson Education, Inc.

```
StackInterface<String> stringStack = new OurStack<>();  
(a) stringStack.push("Jim");  
(b) stringStack.push("Jess");  
(c) stringStack.push("Jill");  
(d) stringStack.push("Jane");  
(e) stringStack.push("Joe");  
(f) stringStack.pop();  
(g) stringStack.pop();
```



Question

- Assume a simple stack for integers.

```
Stack s = new Stack();
```

```
s.push(12);
```

```
s.push(4);
```

```
s.push( s.peek() + 2 );
```

```
s.pop();
```

```
s.push( s.peek() );
```

```
//what are contents of stack?
```



Question

- Write a method to print out contents of stack in reverse order.

```
Stack s = new Stack();  
// put stuff in stack  
for(int i = 0; i < 5; i++)  
    s.push( i );  
  
// print out contents of stack  
for(int i = 0; i < s.size(); i++)  
    System.out.print( s.pop() + " ");  
// What is output?
```



Question

- What is output of code on previous slide?
 - A 0 1 2 3 4
 - B 4 3 2 1 0
 - C 4 3 2
 - D 2 3 4
 - E No output due to runtime error.



Question

```
int limit = s.size();  
for(int i = 0; i < limit; i++)  
    System.out.print( s.pop() + " ");  
  
//or  
  
//    while( !s.isEmpty() )  
  
//        System.out.println( s.pop() );
```



Client, Implementation, Interface

- Separate interface and implementation.
Ex: stack, queue, bag, priority queue,
 - Benefits
 - Client can't know details of implementation $\Rightarrow$
client has many implementation from which to choose.
 - Implementation can't know details of client needs $\Rightarrow$
many clients can re-use the same implementation
-



Client, Implementation, Interface

Client: program using operations defined in interface.

Implementation: actual code implementing operations.

Interface: description of data type, basic operations.



Stack Design Decision

- When stack is empty
 - What to do with pop and peek?
- Possible actions
 - Assume that the stack is not empty
 - Return null
 - Throw an exception



Interface for the Stack

```
/** An interface for the ADT stack. */
public interface StackInterface<T>
{
    /** Adds a new entry to the top of this stack.
        @param newEntry An object to be added to the stack. */
    public void push(T newEntry);

    /** Removes and returns this stack's top entry.
        @return The object at the top of the stack.
        @throws EmptyStackException if the stack is empty before the
operation. */
    public T pop();

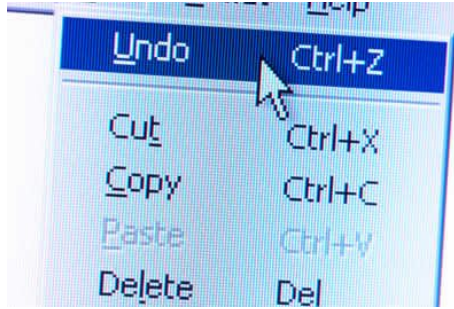
    /** Retrieves this stack's top entry.
        @return The object at the top of the stack.
        @throws EmptyStackException if the stack is empty. */
    public T peek();

    /** Detects whether this stack is
empty.
        @return True if the stack is empty.
    */
    public boolean isEmpty();

    /** Removes all entries from this stack.
    */
    public void clear();
}
// end StackInterface
```



Applications Using Stacks



- Undo in a word processor
 - Back button in a Web browser.
 - The runtime stack used by a process (running program) to keep track of methods in progress.
 - Java virtual machine
 - *and a lot more...*
-



Balanced Symbol Checking

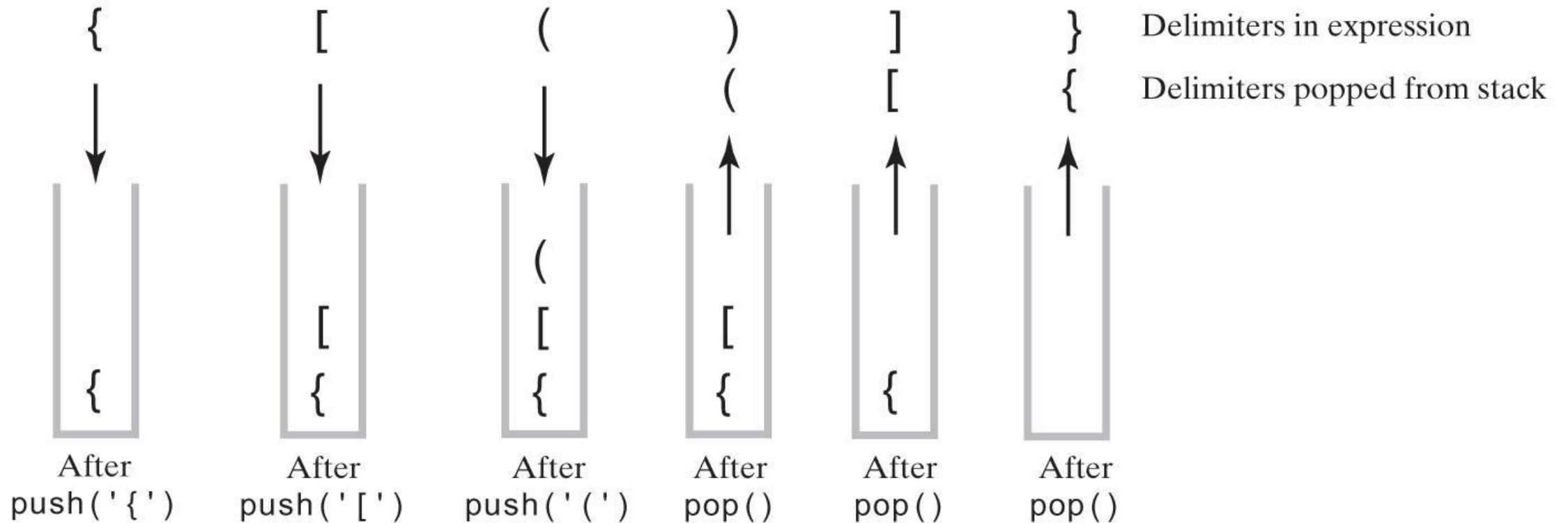
- In processing programs and working with computer languages there are many instances when symbols must be balanced

{ }, [], ()

- Balanced expressions: delimiters paired correctly
 - A stack is useful for checking symbol balance.
 - When a closing symbol is found it must match the most recent opening symbol of the same type.
-



Balanced Symbol Checking



© 2019 Pearson Education, Inc.

The contents of a stack during the scan of an expression that contains the balanced delimiters **{ [()] }**



Balanced Symbol Checking

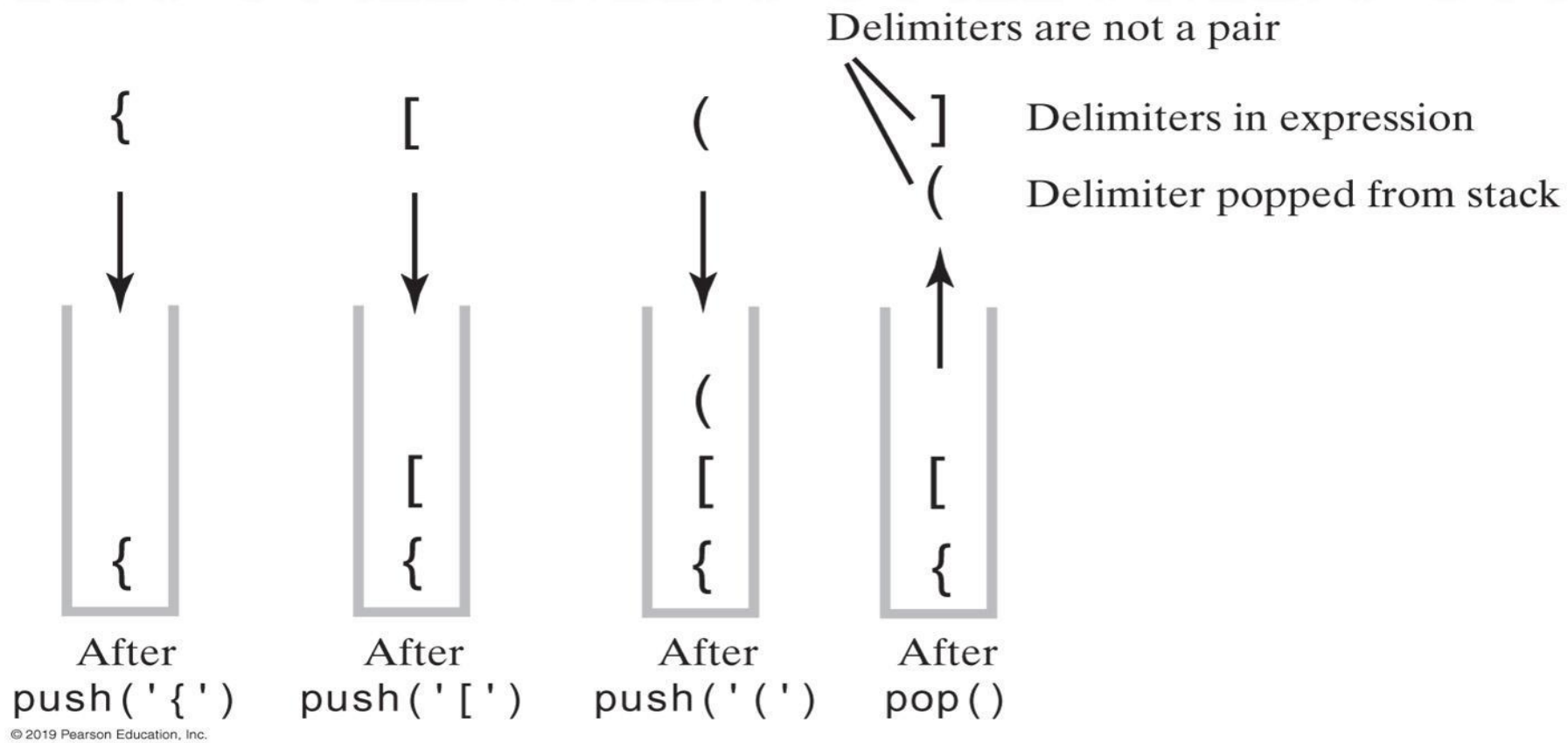


FIGURE 5-4 The contents of a stack during the scan of an expression that contains the **unbalanced** delimiters { [() }



Balanced Symbol Checking

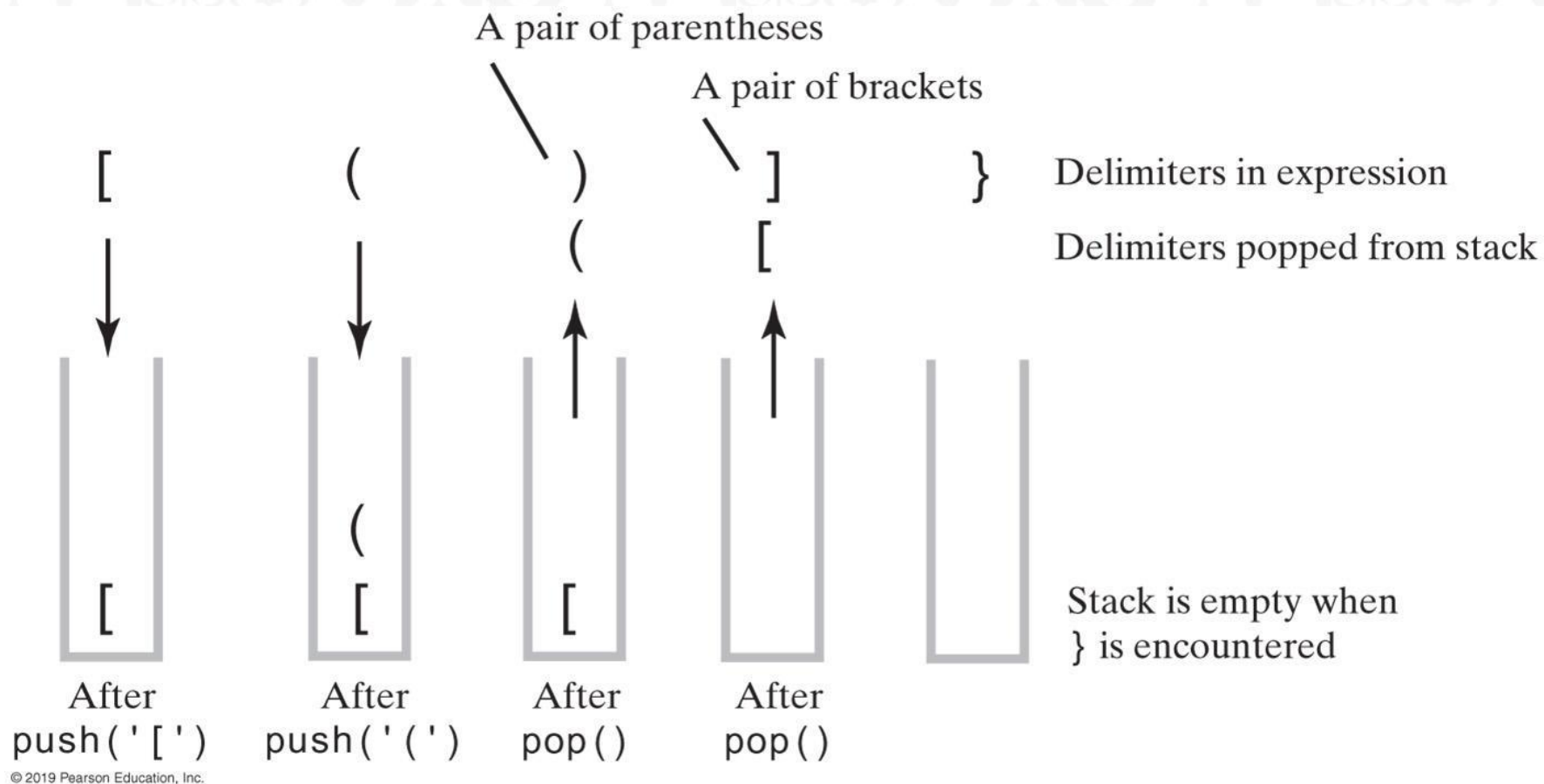


FIGURE 5-5 The contents of a stack during the scan of an expression that contains the **unbalanced delimiters** [()] }



Balanced Symbol Checking

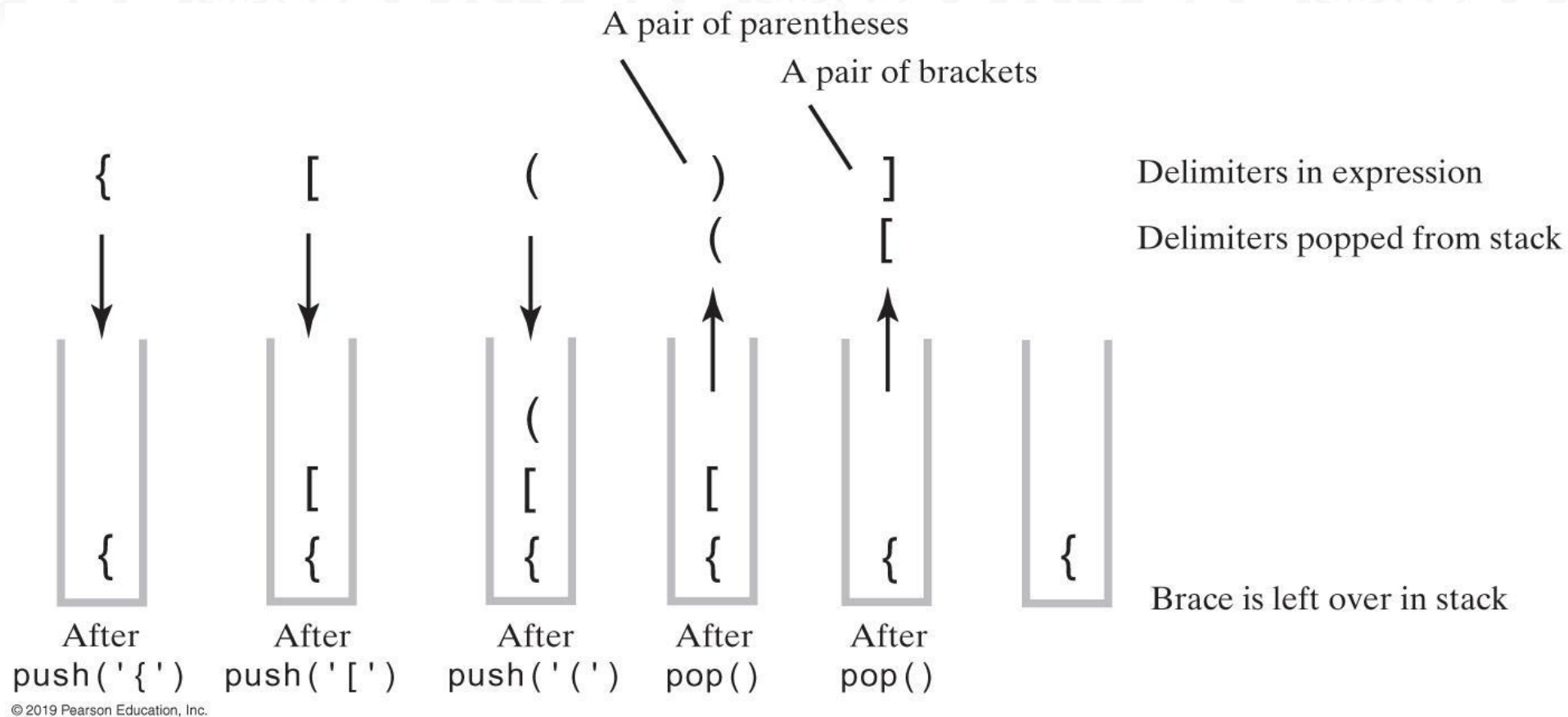


FIGURE 5-6 The contents of a stack during the scan of an expression that contains the **unbalanced delimiters** { [()]



Balanced Symbol Checking

An input expression is balanced if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Algorithm?



Balanced Symbol Checking

```
Algorithm checkBalance(expression)
// Returns true if the parentheses, brackets, and braces in an
expression are paired correctly.
isBalanced = true // The absence of delimiters is balanced
while ((isBalanced == true) and not at end of expression)
{
    nextCharacter = next character in expression
    switch (nextCharacter)
    {
        case '(': case '[': case '{':
            Push nextCharacter onto stack
            break
        case ')': case ']': case '}':
            if (stack is empty)
                isBalanced = false
            else
            {
                openDelimiter = top entry of stack
                Pop stack
                isBalanced = true or false according to whether
openDelimiter and nextCharacter are a pair of delimiters
            }
            break
    }
}
if (stack is not empty)
    isBalanced = false
return isBalanced
```



Implementation of Algorithm (Part 1)

```
/** A class that checks whether the parentheses, brackets, and braces  
    in a string occur in left/right pairs. */
```

```
public class BalanceChecker
```

```
{
```

```
    // Returns true if the given characters, open and close, form a pair  
    // of parentheses, brackets, or braces.
```

```
    private static boolean isPaired(char open, char close)
```

```
{
```

```
        return (open == '(' && close == ')') ||
```

```
               (open == '[' && close == ']') ||
```

```
               (open == '{' && close == '}');
```

```
    } // end isPaired
```

```
/** Decides whether the parentheses, brackets, and braces  
    in a string occur in left/right pairs.
```

```
    @param expression A string to be checked.
```

```
    @return True if the delimiters are paired correctly. */
```

```
    public static boolean checkBalance(String expression)
```

```
{
```

[SEE NEXT SLIDE FOR IMPLEMENTATION]

```
    } // end checkBalance
```

```
} // end BalanceChecker
```



Implementation of Algorithm (Part 2)

```
StackInterface<Character> openDelimiterStack = new LinkedStack<>();
int characterCount = expression.length();
boolean isBalanced = true;
int index = 0;
char nextCharacter = ' ';

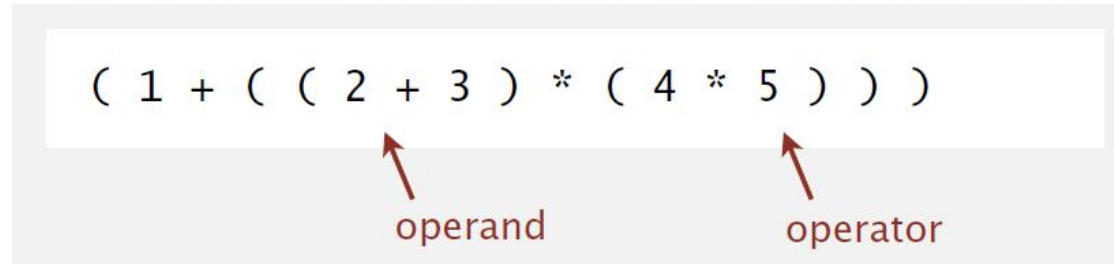
while (isBalanced && (index < characterCount)) {
    nextCharacter = expression.charAt(index);
    switch (nextCharacter) {
        case '(': case '[': case '{':
            openDelimiterStack.push(nextCharacter);
            break;
        case ')': case ']': case '}':
            if (openDelimiterStack.isEmpty())
                isBalanced = false;
            else {
                char openDelimiter = openDelimiterStack.pop();
                isBalanced = isPaired(openDelimiter, nextCharacter);
            } // end if
            break;
        default: break; // Ignore unexpected characters
    } // end switch
    index++;
} // end while

if (!openDelimiterStack.isEmpty())
    isBalanced = false;

return isBalanced;
} // end checkBalance
```



Arithmetic Expression Evaluation



- The precedence of operators affects the order of operations.
 - How to evaluate an arithmetic expression?
-



Arithmetic Expression Evaluation

- Two-stack algorithm. [E. W. Dijkstra]

Value: push onto the value stack.

Operator: push onto the operator stack.

Left parenthesis: ignore.

Right parenthesis: pop operator and two values; push the result of applying that operator to those values onto the operand stack.

value stack

operator stack

(1 + ((2 + 3) * (4 * 5)))

operand

operator



Arithmetic Expression Evaluation

```
public class Evaluate
{
    public static void main(String[] args)
    {
        Stack<String> ops = new Stack<String>();
        Stack<Double> vals = new Stack<Double>();
        while (!StdIn.isEmpty()) {
            String s = StdIn.readString();
            if (s.equals("(")) ;
            else if (s.equals("+")) ops.push(s);
            else if (s.equals("*")) ops.push(s);
            else if (s.equals(")"))
            {
                String op = ops.pop();
                if (op.equals("+")) vals.push(vals.pop() + vals.pop());
                else if (op.equals("*")) vals.push(vals.pop() * vals.pop());
            }
            else vals.push(Double.parseDouble(s));
        }
        StdOut.println(vals.pop());
    }
}
```

```
% java Evaluate
( 1 + ( ( 2 + 3 ) * ( 4 * 5 ) ) )
101.0
```



Arithmetic Expression Evaluation

- Why correct?

When algorithm encounters an operator surrounded by two values within parentheses, it leaves the result on the value stack.

$$(1 + ((2 + 3) * (4 * 5)))$$

as if the original input were:

$$(1 + (5 * (4 * 5)))$$

Repeating the argument:

$$\begin{aligned} & (1 + (5 * 20)) \\ & (1 + 100) \\ & 101 \end{aligned}$$

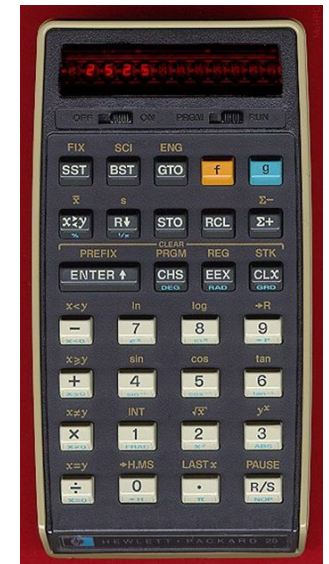


Processing Algebraic Expressions

- **Infix:** each binary operator appears between its operands
 $a + b$
 - **Prefix:** each binary operator appears before its operands
 $+ a b$
 - **Postfix:** each binary operator appears after its operands
 $a b +$
-



- 1 2 3 4 ^ * +





Question

- What does the following postfix expression evaluate to?

6 3 2 + *

A: 18

B: 36

C: 24

D: 11

E: 30



Evaluating Postfix Expressions

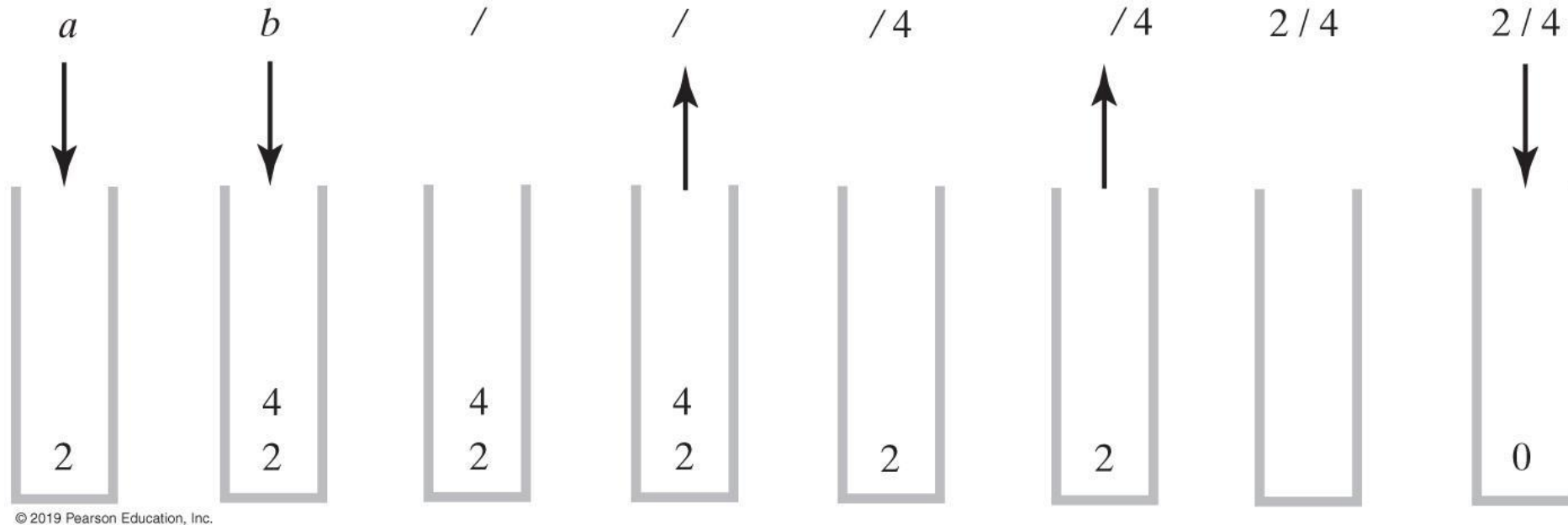


FIGURE 5-10 The stack during the evaluation of the postfix expression $a \ b \ /$ when a is 2 and b is 4



Evaluating Postfix Expressions

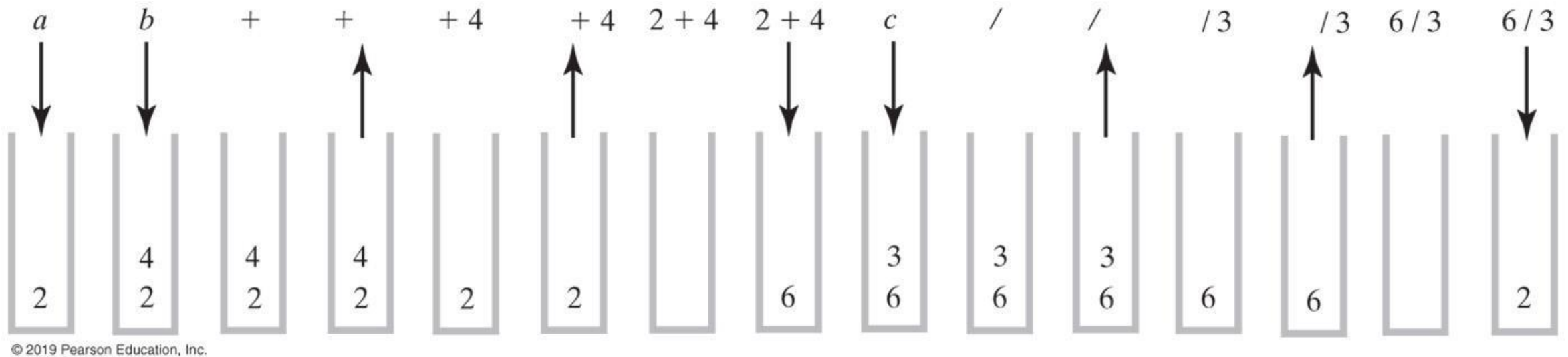


FIGURE 5-11 The stack during the evaluation of the postfix expression $a \ b \ + \ c \ /$ when a is 2, b is 4, and c is 3



Evaluating Postfix Expressions

Algorithm evaluatePostfix(postfix)

// Evaluates a postfix expression.

valueStack = a new empty stack

while (postfix has characters left to parse)

{

nextCharacter = next nonblank character of postfix

switch (nextCharacter)

{

case variable:

valueStack.push(value of the variable nextCharacter)

break

case '+' : case '-' : case '' : case '/' : case '^' :*

operandTwo = valueStack.pop()

operandOne = valueStack.pop()

result = the result of the operation in nextCharacter and

its operands operandOne and operandTwo

valueStack.push(result)

break

default: break // Ignore unexpected characters

}

}

return valueStack.peek()

**Algorithm for
evaluating postfix
expressions.**



Processing Algebraic Expressions

Infix

$$a + b$$

$$(a + b) * c$$

$$a + b * c$$

Postfix

$$ab +$$

$$ab + c *$$

$$abc * +$$



Converting Infix to Postfix

$a + b * c$

Next Character in Infix Expression	Postfix Form	Operator Stack (bottom to top)
a	a	
$+$	a	$+$
b	$a b$	$+$
$*$	$a b$	$+ *$
c	$a b c$	$+ *$
	$a b c *$	$+$
	$a b c * +$	

$a ^ b ^ c$

Next Character in Infix Expression	Postfix Form	Operator Stack (bottom to top)
a	a	
$^$	a	$^$
b	$a b$	$^$
$^$	$a b$	$^ ^$
c	$a b c$	$^ ^$
	$a b c ^$	$^$
	$a b c ^ ^$	

$a - b + c$

Next Character in Infix Expression	Postfix Form	Operator Stack (bottom to top)
a	a	
$-$	a	$-$
b	$a b$	$-$
$+$	$a b -$	
	$a b -$	$+$
c	$a b - c$	$+$
	$a b - c +$	



Converting Infix to Postfix

*a / b * (c + (d - e))*

Next Character from Infix Expression	Postfix Form	Operator Stack (bottom to top)
<i>a</i>	<i>a</i>	
<i>/</i>	<i>a</i>	<i>/</i>
<i>b</i>	<i>a b</i>	<i>/</i>
<i>*</i>	<i>a b /</i>	
<i>(</i>	<i>a b /</i>	<i>*</i>
<i>c</i>	<i>a b /</i>	<i>* (</i>
<i>+</i>	<i>a b / c</i>	<i>* (</i>
<i>(</i>	<i>a b / c</i>	<i>* (+</i>
<i>d</i>	<i>a b / c</i>	<i>* (+ (</i>
<i>-</i>	<i>a b / c d</i>	<i>* (+ (</i>
<i>e</i>	<i>a b / c d</i>	<i>* (+ (-</i>
<i>)</i>	<i>a b / c d e</i>	<i>* (+ (-</i>
<i>)</i>	<i>a b / c d e -</i>	<i>* (+ (</i>
<i>)</i>	<i>a b / c d e -</i>	<i>* (+</i>
<i>)</i>	<i>a b / c d e - +</i>	<i>* (</i>
	<i>a b / c d e - +</i>	<i>*</i>
	<i>a b / c d e - + *</i>	



Infix-to-postfix Conversion

- **Key Observations:**

Operand	Append each operand to the end of the output expression.
Operator ^	Push ^ onto the stack.
Operator +, -, *, or /	Pop operators from the stack, appending them to the output expression, until either the stack is empty or its top entry has a lower precedence than the newly encountered operator . Then push the new operator onto the stack.
Open parenthesis	Push (onto the stack.
Close parenthesis	Pop operators from the stack and append them to the output expression until an open parenthesis is popped . Discard both parentheses.



Converting Infix to Postfix

$a + b * (c + d)$

Animation

a



#1

a



#2

ab



#3

ab



#4

ab



#5

abc



#6

abc



#7

abcd



#8

abcd+



#9

abcd+



#10

abcd+*



#11

abcd+*+



#12



Pseudocode of Converting infix to postfix

```
Algorithm convertToPostfix(infix)
// Converts an infix expression to an equivalent postfix expression.
operatorStack = a new empty stack
postfix = a new empty string
while (infix has characters left to parse)
{
    nextCharacter = next nonblank character of infix
    switch (nextCharacter)
    {
        case variable:
            Append nextCharacter to postfix
            break
        case '^' :
```



Pseudocode of Converting infix to postfix

```
    operatorStack.push(nextCharacter)
    break
case '+' : case '-' : case '*' : case '/' :
    while (!operatorStack.isEmpty() and
           precedence of nextCharacter <= precedence of
operatorStack.peek())
    {
        Append operatorStack.peek() to postfix
        operatorStack.pop()
    }
    operatorStack.push(nextCharacter)
    break
case '(' :
    operatorStack.push(nextCharacter)
    break
case ')' : // Stack is not empty if infix expression is valid
    topOperator = operatorStack.pop()
    while (topOperator != '(')
    {
        Append topOperator to postfix
        topOperator = operatorStack.pop()
    }
    break
default: break // Ignore unexpected characters
}
}
```

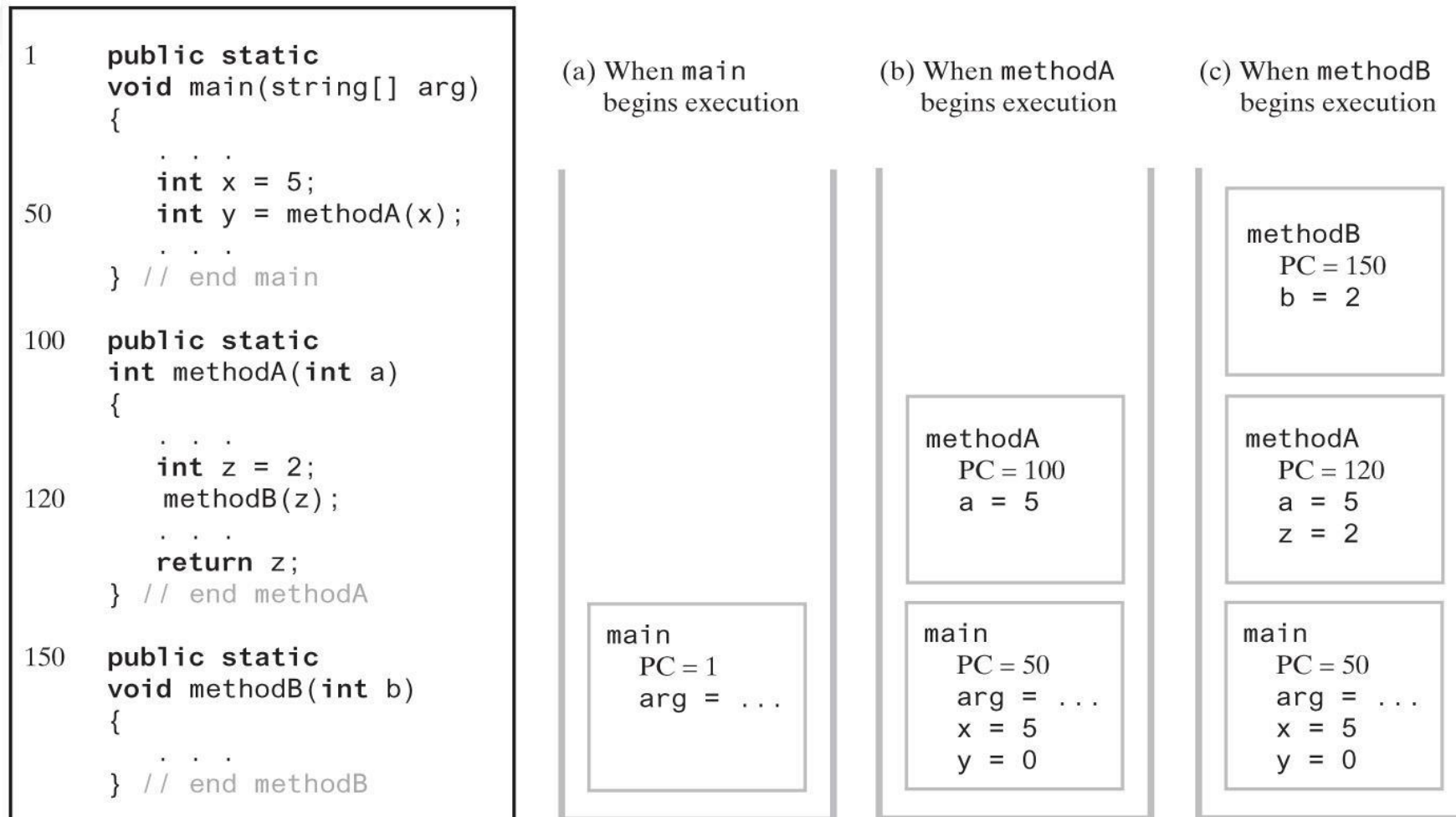


Pseudocode of Converting infix to postfix

```
while (!operatorStack.isEmpty())  
{  
    topOperator = operatorStack.pop()  
    Append topOperator to postfix  
}  
return postfix
```



The Application Program Stack



Program

Program stack at three points in time (PC is the program counter)

© 2019 Pearson Education, Inc.

FIGURE 5-13 The program stack as a program executes



Java Class Library: The Class Stack

- Found in `java.util.Stack`
- Methods
 - A constructor – creates an empty stack
 - `public T push(T item);`
 - `public T pop();`
 - `public T peek();`
 - `public boolean empty();`

```
import java.util.Stack;

public class Main {
    public static void main(String[] args) {
        Stack<String> pile = new Stack<>();

        pile.push("Zasmin");
        pile.push("Zamora");

        while (!pile.empty()) {
            System.out.println(pile.pop());
        }
    }
}
```